

SQLX & Migrations

I. Lesson

When working with backend with Rust, you'll have to handle **migrations**.

A **migration** is a way to update the structure of a database, like adding or changing tables and columns. It helps keep the database in sync with your code as your app changes.

... And for that, **SQLx** is the best.

What is SQLx? 🤔

SQLx

- Migrations: Handles migrations for you.
- Type safe: Thanks to `query_as!` macros.
- Raw SQL: You can write raw SQL safely (more efficient).
- Databases: Supports SQLite, MySQL, Postgresql.
- Fully asynchronous.

Installation

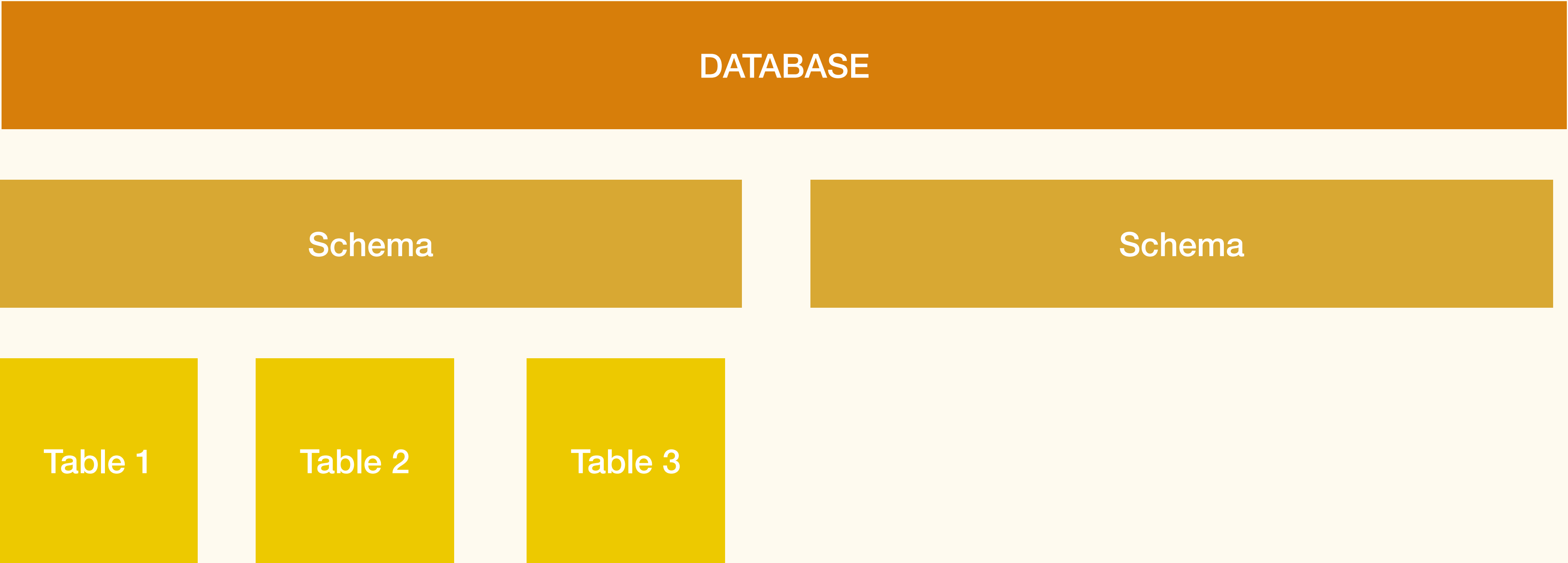


```
cargo install sqlx-cli
```

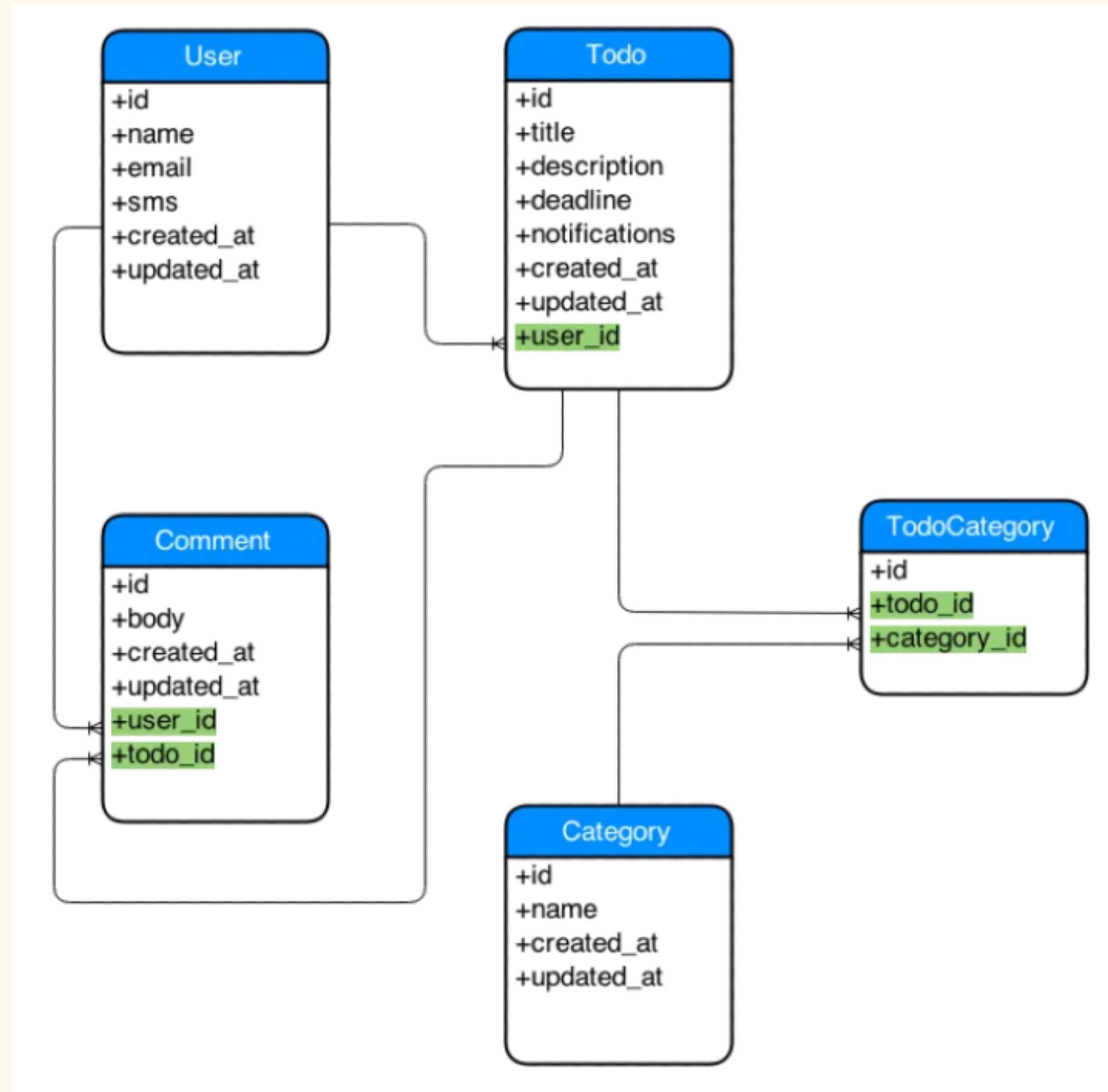
You need to have PostgreSQL installed



💡 A few reminders about Databases



Several tables linked
with FK.



Create a DB



```
CREATE DATABASE my_awesome_db;
```

psql - List DB

```
postgres=# \l
postgres=# \l

                                List of databases
  Name          | Owner   | Encoding | Locale Provider | Collate | Ctype | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
airbnb_clone    | postgres | UTF8      | libc             | C       | C     |             |             |
app_shortener   | postgres | UTF8      | libc             | C       | C     |             |             |
everlabs        | postgres | UTF8      | libc             | C       | C     |             |             |
postgres        | user     | UTF8      | libc             | C       | C     |             |             | =Tc/user +
                |          |           |                  |         |       |             |             | user=CTc/user +
                |          |           |                  |         |       |             |             | postgres=CTc/user
template0       | user     | UTF8      | libc             | C       | C     |             |             | =c/user +
                |          |           |                  |         |       |             |             | user=CTc/user
template1       | user     | UTF8      | libc             | C       | C     |             |             | =c/user +
                |          |           |                  |         |       |             |             | user=CTc/user
w3_d4_axum_todos | postgres | UTF8      | libc             | C       | C     |             |             |
w5_fullstack_auth | postgres | UTF8      | libc             | C       | C     |             |             |
w5_fullstack_bookstore | postgres | UTF8      | libc             | C       | C     |             |             |
w5_fullstack_bookstore_day_1 | postgres | UTF8      | libc             | C       | C     |             |             |
w5_fullstack_bookstore_day_2 | postgres | UTF8      | libc             | C       | C     |             |             |
w5_fullstack_todos | postgres | UTF8      | libc             | C       | C     |             |             |
(12 rows)

postgres=# █
```

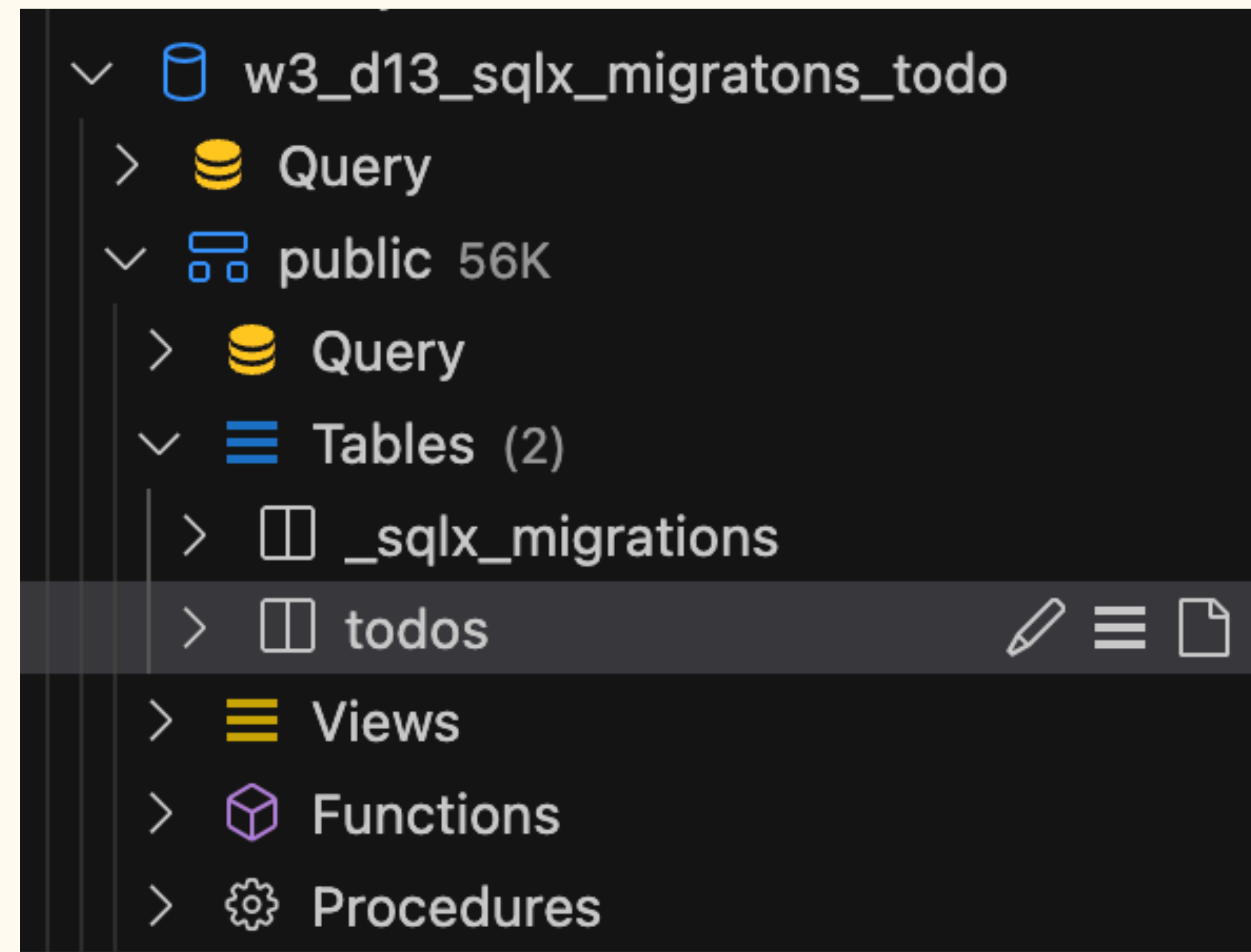
%K to generate a command

A DB has **schemas** and **tables**

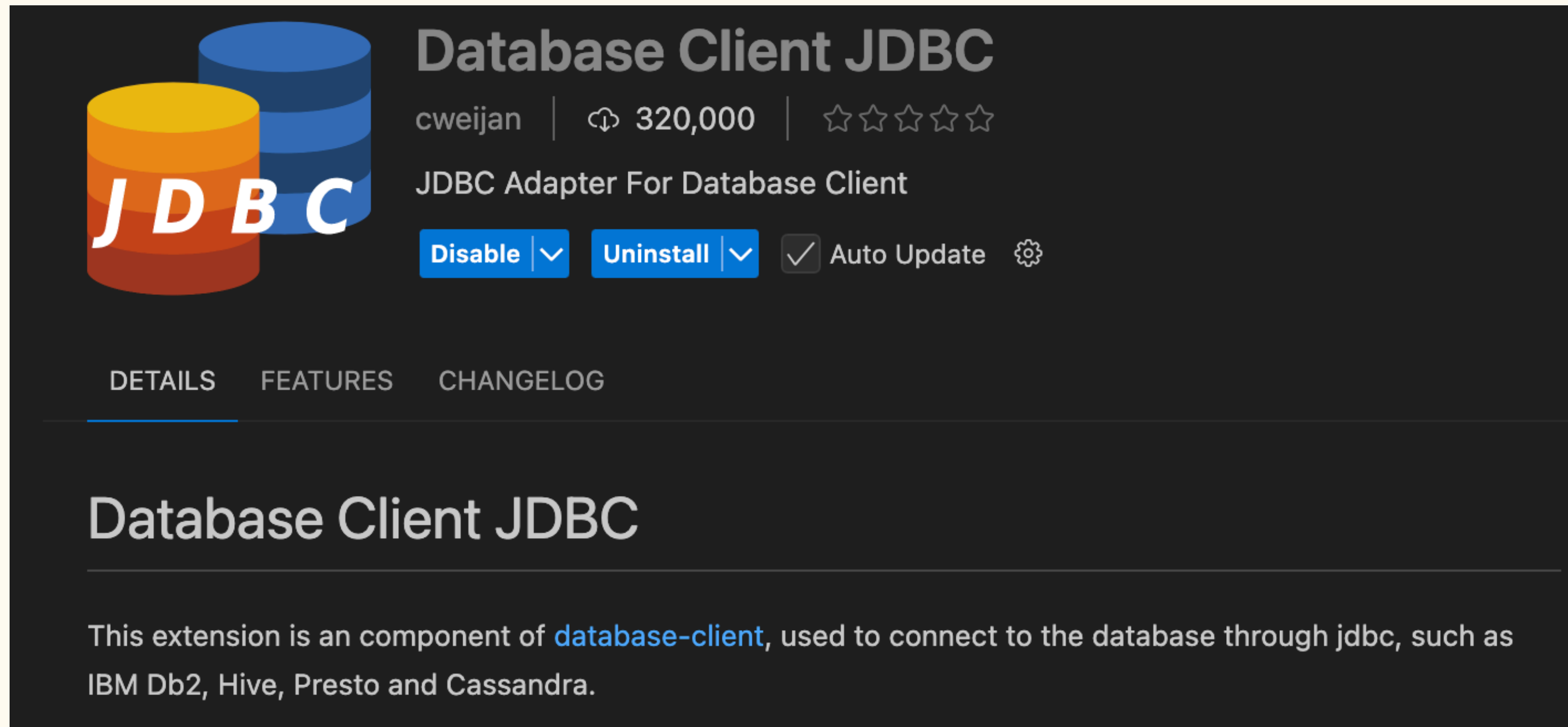
Database ->

Schema ->

Tables ->



VS Code extension



The screenshot shows the VS Code extension marketplace interface for the 'Database Client JDBC' extension. The extension is by 'cweijan' and has 320,000 downloads and a 5-star rating. The description states it is a 'JDBC Adapter For Database Client'. The interface includes buttons for 'Disable', 'Uninstall', and 'Auto Update' (which is checked). Below the extension name, there are tabs for 'DETAILS', 'FEATURES', and 'CHANGELOG'. The 'DETAILS' tab is selected, showing the extension's name and a description: 'This extension is an component of [database-client](#), used to connect to the database through jdbc, such as IBM Db2, Hive, Presto and Cassandra.'

Database Client JDBC
cweijan | 320,000 | ★★★★★
JDBC Adapter For Database Client

Disable | Uninstall | ☒ Auto Update ⚙️

DETAILS | FEATURES | CHANGELOG

Database Client JDBC

This extension is an component of [database-client](#), used to connect to the database through jdbc, such as IBM Db2, Hive, Presto and Cassandra.

II. Project

We will create a simple Todo App using SQLx.

Add a new migration



```
sqlx migrate add create_todo_table
```

Create a table



```
CREATE TABLE IF NOT EXISTS todos (  
    id SERIAL PRIMARY KEY,  
    text VARCHAR(255) NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```


AppState

```
● ● ●

use sqlx::PgPool;

#[derive(Clone)]
pub struct AppState {
    pub pool: PgPool,
}

pub async fn create_db_pool() -> sqlx::Result<PgPool> {
    let database_url = std::env::var("DATABASE_URL").expect("DATABASE_URL must be set");
    let pool = PgPool::connect(&database_url).await?;

    // Run migrations
    sqlx::migrate!("./migrations").run(&pool).await?;

    Ok(pool)
}
```

.env



```
DATABASE_URL=postgresql://test_user:password@localhost:5432/w3_d13_sqlx_migratons_todo
```

Routes



```
pub fn todos_routes() -> Router<AppState> {  
    Router::new()  
        .route("/todos", get(Todo::get_all).post(Todo::create))  
        .route(  
            "/todos/{id}",  
            get(Todo::get_by_id).patch(Todo::update).delete(Todo::delete),  
        )  
}
```


Struct



```
#[derive(Debug, Serialize, FromRow)]  
pub struct Todo {  
    pub id: i32,  
    pub text: String,  
    pub created_at: OffsetDateTime,  
    pub updated_at: OffsetDateTime,  
}
```

Handlers

```
pub async fn get_all(app_state: State<AppState>) -> Result<Json<Vec<Todo>>, TodoError> {  
    let todos = sqlx::query_as!(  
        Todo,  
        r#"  
            SELECT *  
            FROM todos  
            ORDER BY created_at DESC  
        "#  
    )  
    .fetch_all(&app_state.pool)  
    .await?;  
  
    Ok(Json(todos))  
}
```


thiserror crate



```
#[derive(Debug, thiserror::Error)]
pub enum TodoError {
    #[error("Todo with id {0} not found")]
    NotFound(i32),
    #[error("Database error: {0}")]
    Database(#[from] sqlx::Error),
}
```

Simple Todo app with SQLx

Your turn!